VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT
on

Machine Learning (23CS6PCMAL)

Submitted by

Harshitha H G (1BM23CS108)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Sep-2024 to Jan-2025

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "Machine Learning (23CS6PCMAL)" carried out by **Harshitha H G (1BM22CS108),** who is bonafide student of **B.M.S. College of Engineering.** It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Machine Learning (23CS6PCMAL) work prescribed for the said degree.

Dr. Chaitra D Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE

Index

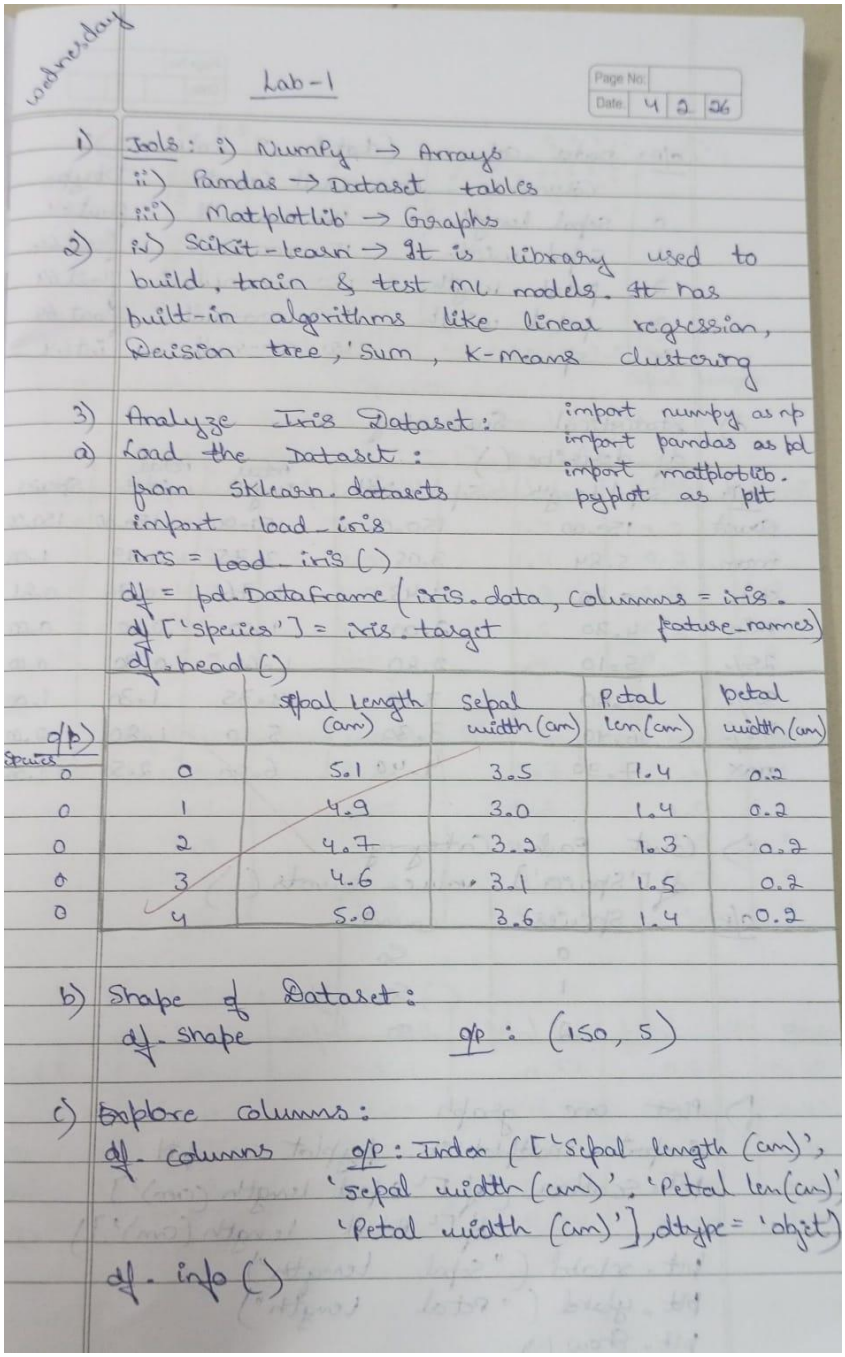
Sl. No.	Date	Experiment Title	Page No.
1	21-2-2025	Write a python program to import and export data using Pandas library functions	4-6
2	3-3-2025	Demonstrate various data pre-processing techniques for a given dataset	7-11
3	10-3-2025	Implement Linear and Multi-Linear Regression algorithm using appropriate dataset	12-17
4	17-3-2025	Build Logistic Regression Model for a given dataset	18-22
5	24-3-2025	Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.	23-27
6	7-4-2025	Build KNN Classification model for a given dataset.	28-29
7	21-4-2025	Build Support vector machine model for a given dataset	30-32
8	5-5-2025	Implement Random forest ensemble method on a given dataset.	31-34
9	5-5-2025	Implement Boosting ensemble method on a given dataset.	35-36
10	12-5-2025	Build k-Means algorithm to cluster a set of data stored in a .CSV file.	37-39
11	12-5-2025	Implement Dimensionality reduction using Principal Component Analysis (PCA) method.	40-44

Github Link:
https://github.com/Harshithahrgopal/ML-Lab

Program 1

Write a python program to import and export data using Pandas library functions

Screenshot



Wednesday

Lab-1

Page No:
Date: 4 2 26

1) Tools: i) Numpy → Arrays
ii) Pandas → Dataset tables
iii) Matplotlib → Graphs
2) iv) Scikit-learn → It is library used to build, train & test ML models. It has built-in algorithms like linear regression, Decision tree, SVM, K-means clustering

3) Analyze Iris Dataset:
a) Load the Dataset:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets
import load_iris
iris = load_iris()
df = pd.DataFrame(iris.data, columns = iris.feature_names)
df['species'] = iris.target
df.head()
```

o/p)

		sepal length (cm)	sepal width (cm)	Petal len (cm)	petal width (cm)
species					
0	0	5.1	3.5	1.4	0.2
0	1	4.9	3.0	1.4	0.2
0	2	4.7	3.2	1.3	0.2
0	3	4.6	3.1	1.5	0.2
0	4	5.0	3.6	1.4	0.2

b) Shape of Dataset:
df.shape o/p: (150, 5)

c) Explore columns:
df.columns o/p: Index(['Sepal length (cm)', 'sepal width (cm)', 'Petal len(cm)', 'Petal width (cm)'], dtype='object')
df.info()

o/p: Data columns (total 5 columns):

	Column	Non-Null Count	Dtype
0	sepal length	150 non-null	float64
1	sepal width	150 non-null	float64
2	petal length	150 non-null	float64
3	petal width	150 non-null	float64
4	species	150 non-null	int64

d) Statistical Summary

df.describe()

o/p:	sepal length	sepal width	Petal length	Petal width	Species
count	150.00	150.0	150.00	150.00	150.00
mean	5.84	3.05	3.75	1.19	1.00
std	0.82	0.43	1.76	0.76	0.81
min	4.30	2.00	1.00	0.10	0.00
25%	5.10	2.80	1.60	0.30	0.00
50%	5.80	3.00	4.35	1.30	1.00
75%	6.40	3.30	5.10	1.80	2.00
max	7.90	4.40	6.90	2.50	2.00

e) Count Each Category

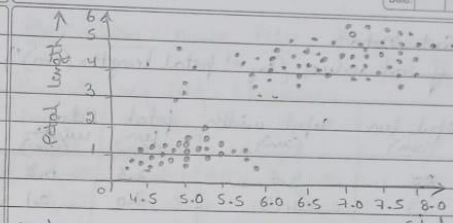
df['Species'].value_counts()

o/p:

Species	Count
0	50
1	50
2	50

f) Plot one graph

```
import matplotlib.pyplot as plt
plt.scatter(df['sepal length (cm)'],
            df['Petal length (cm)'])
plt.xlabel("Sepal length")
plt.ylabel("Petal length")
plt.show()
```



g) Compare Two Species

df[df['Species'] == 0].head()

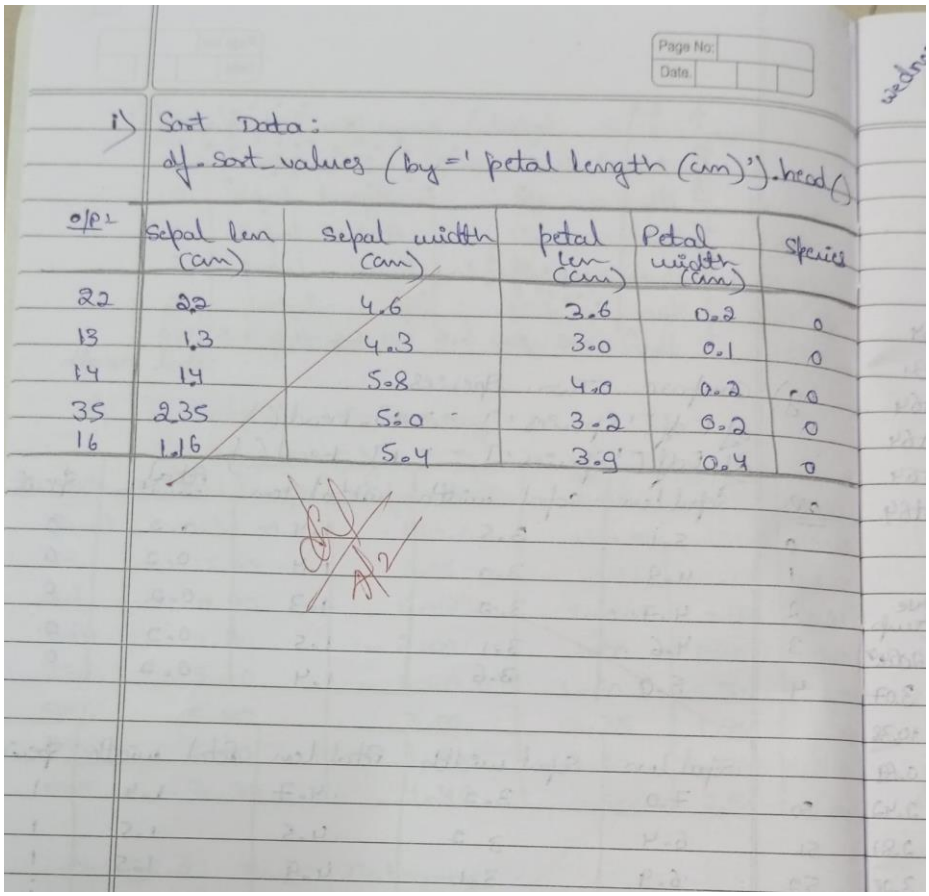
df[df['Species'] == 1].head()

o/p:	sepal len	sepal width	Petal len	Petal width	Species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

	Sepal len	Sepal width	Petal len	Petal width	Species
50	7.0	3.2	4.7	1.4	1
51	6.4	3.2	4.5	1.5	1
52	6.9	3.1	4.9	1.5	1
53	5.5	2.3	4.0	1.3	1
54	6.5	2.8	4.6	1.5	1

h) Correlation: df.corr()

	sepal len	sepal width	petal len	petal width	Species
sepal len	1.00	-0.11	0.87	0.81	0.78
sepal width	-0.11	1.00	-0.42	-0.36	-0.42
Petal len	0.87	-0.42	1.00	0.96	0.94
Petal width	0.81	-0.36	0.96	1.00	0.95
species	0.78	-0.42	0.94	0.95	1.00



1) Sort Data:

df.sort_values(by='petal length (cm)').head()

o/p:

	Sepal len (cm)	Sepal width (cm)	petal len (cm)	Petal width (cm)	Species
22	2.2	4.6	3.6	0.2	0
13	1.3	4.3	3.0	0.1	0
14	1.4	5.8	4.0	0.2	0
35	2.35	5.0	3.2	0.2	0
16	1.16	5.4	3.9	0.4	0

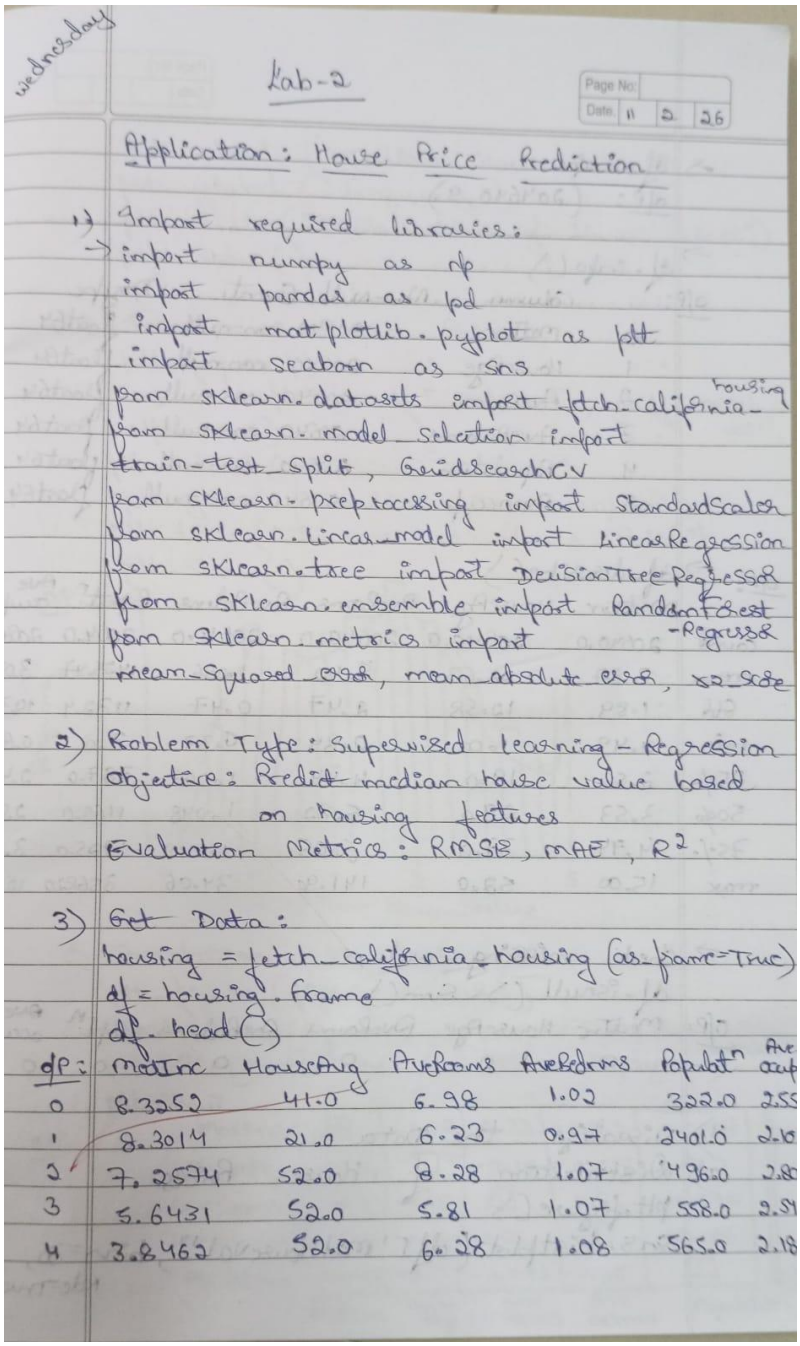
Code:

```
import pandas as pd

data = pd.read_csv('input.csv')

print(data)

print(data.head())

data.to_csv('output.csv', index=False)

print("Data successfully exported to 'output.csv'")
```

Program 2

Demonstrate various data pre-processing techniques for a given dataset

Screenshot



wednesday

Lab-2

Page No:
Date: 11 2 26

Application: House Price Prediction

1) Import required libraries:

```
-> import numpy as np
   import pandas as pd
   import matplotlib.pyplot as plt
   import seaborn as sns
   from sklearn.datasets import fetch_california_housing
   from sklearn.model_selection import
   train_test_split, GridSearchCV
   from sklearn.preprocessing import StandardScaler
   from sklearn.linear_model import LinearRegression
   from sklearn.tree import DecisionTreeRegressor
   from sklearn.ensemble import RandomForestRegressor
   from sklearn.metrics import
   mean_squared_error, mean_absolute_error, r2_score
```

2) Problem Type: Supervised learning - Regression
Objective: Predict median house value based on housing features
Evaluation Metrics: RMSE, MAE, R^2

3) Get Data:

```
housing = fetch_california_housing(as_frame=True)
df = housing.frame
df.head()
```

df:

	MedInc	HouseAvg	AveRooms	AveBedrms	Populat^n	Ave occup
0	8.3252	41.0	6.98	1.02	322.0	2.55
1	8.3014	21.0	6.23	0.97	2401.0	2.10
2	7.2574	52.0	8.28	1.07	496.0	2.80
3	5.6431	52.0	5.81	1.07	558.0	2.51
4	3.8462	52.0	6.28	1.08	565.0	2.18

→ df.shape

o/p: (20640, 9)

→ df.info()

o/p:

	Column	Non-Null Count	Dtype
0	MedInc	20640 non-null	float64
1	HouseAge	20640 non-null	float64
2	AveRooms	20640 non-null	float64
3	AveBedrms	20640 non-null	float64
4	Population	20640 non-null	float64
5	AveOccup	20640 non-null	float64

o/p: → df.describe()

	MedInc	HouseAge	AveRooms	AveBedrms	Popln	Ave occup
count	20640.0	20640.0	20640.0	2064.0	2064.0	2064.0
mean	3.87	28.63	5.43	1.096	1425.47	3.07
std	1.89	12.58	2.47	0.47	1132.4	10.38
min	0.49	1.00	0.84	0.33	3.0	0.69
25%	2.56	18.0	4.44	1.006	787.0	2.42
50%	3.53	29.0	5.22	1.048	1166.0	2.81
75%	4.74	37.0	6.05	1.099	1725.0	3.28
max	15.00	52.0	141.90	34.06	35682.0	1243.33

→ Check missing values

df.isnull().sum()

o/p:

MedInc	HouseAge	AveRooms	AveBedrms	Popln	Ave occup
0	0	0	0	0	0

4) Visualize the Data

a) Distribution of House Prices

plt.figure()

sns.histplot(df['medHouseVal'], bins=30, kde=True)

plt.xlabel("Median House Value")

plt.ylabel("Frequency")

plt.title("Distribution of House Prices")

plt.show()

b) Income vs House Price

plt.figure()

sns.scatterplot(x=df['MedInc'], y=df['medHouseVal'], alpha=0.5)

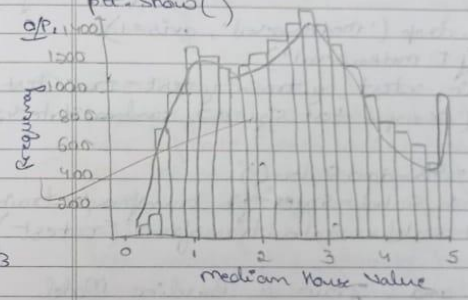
plt.xlabel("Median Income")

plt.ylabel("Median House value")

plt.title("Income vs House Price")

plt.show()

o/p:



c) Correlation Heatmap

plt.figure(figsize=(10,8))

sns.heatmap(df.corr(), annot=True, cmap='coolwarm')

plt.title("Feature Correlation Heatmap")

plt.show

o/p:

	MedInc	HouseAge	AveRooms	AveBedrms	Population
MedInc	1	-0.12	0.33	-0.062	0.0048
HouseAge	-0.12	1	-0.15	-0.078	-0.3
AveRooms	0.33	-0.15	1	0.85	-0.072
AveBedrms	-0.062	-0.07	0.85	1	-0.066
Population	0.0048	-0.3	-0.072	-0.66	1

d) location based visualization

```
plt.figure()
plt.scatter(df['longitude'], df['latitude'],
    c= df['medHouseVal'], cmap='viridis', s=5)
plt.colorbar(label="House Value")
plt.xlabel("longitude")
plt.ylabel("latitude")
plt.title("House Prices by location")
plt.show()
```

5) Prepare the Data

```
df['RoomsPerHousehold'] = df['AveRooms'] / df['HouseAge']
```

→
```
x=df.drop('medHouseVal', axis=1)
y=df['medHouseVal']
x_train, x_test, y_train, y_test= train_test_split(x, y, test_size=0.2, random_state=42)
```

→
```
Scaler = StandardScaler()
x_train_Scaled = Scaler.fit_transform(x_train)
x_test_scaled = Scaler.transform(x_test)
```

6) Select and Train a Baseline Model

a) linear Regression:

```
lin_reg = LinearRegression()
lin_reg.fit(x_train_Scaled, y_train)
y_pred = lin_reg.predict(x_test_scaled)
rmse_lr = np.sqrt(mean_squared_error(y_test, y_pred))
r2_lr = r2_score(y_test, y_pred)
rmse_lr, r2_lr, mae_lr
mae_lr= mean_absolute_error(y_test, y_pred)
```

7) Train Advanced models

a) Decision tree Regressor:

```
dt=DecisionTreeRegressor(random_state=42)
dt.fit(x_train, y_train)
y_pred = dt.predict(x_test)
rmse_dt= np.sqrt(mean_squared_error(y_test, y_pred))
rmse_dt
```

b) Random Forest Regressor

```
rf = RandomForestRegressor(random_state=42)
rf.fit(x_train, y_train)
y_pred = rf.predict(x_test)
rmse_rf= np.sqrt(mean_squared_error(y_test, y_pred))
rmse_rf
```

8) Fine-Tune the model

```
param_grid = {'n_estimators': [50,100],
'max_depth': [None,10,20] 'min_sample': [2,5]}
grid_search = GridSearchCV(rf, param_grid,
cv=3, scoring='neg_mean_squared_error')
grid_search.fit(x_train, y_train)
best_model= grid_search.best_estimator
y_pred_best = best_model.predict(x_test)
rmse_best = np.sqrt(mean_squared_error(y_test, y_pred))
rmse_best
```

9) Model Comparison

```
results= pd.DataFrame({'Model':['linear Regression', 'Decision Tree', 'Random Forest', 'Tuned Random Forest'],
'RMSE': [rmse_lr, rmse_dt, rmse_rf, rmse_best]
})
results
```

o/p:

	Model	RMSE
0	linear Regression	0.7449
1	Decision Tree	0.7138
2	Random forest	0.5128
3	Tuned Random Forest	0.5132

10) Final Conclusion:

a) which model performed best?

→ Tuned Random forest

b) why did Random forest outperform linear Regression?

→ Handles non-linear relationships

→ Reduces overfitting

→ Combines multiple trees

c) How did tuning improve performance?

→ Selecting optimal depth

→ choosing best number of trees

d) what additional data could further improve predictions?

→ School ratings

→ Crime rate

→ Proximity to transport

→ Amenities

Code:

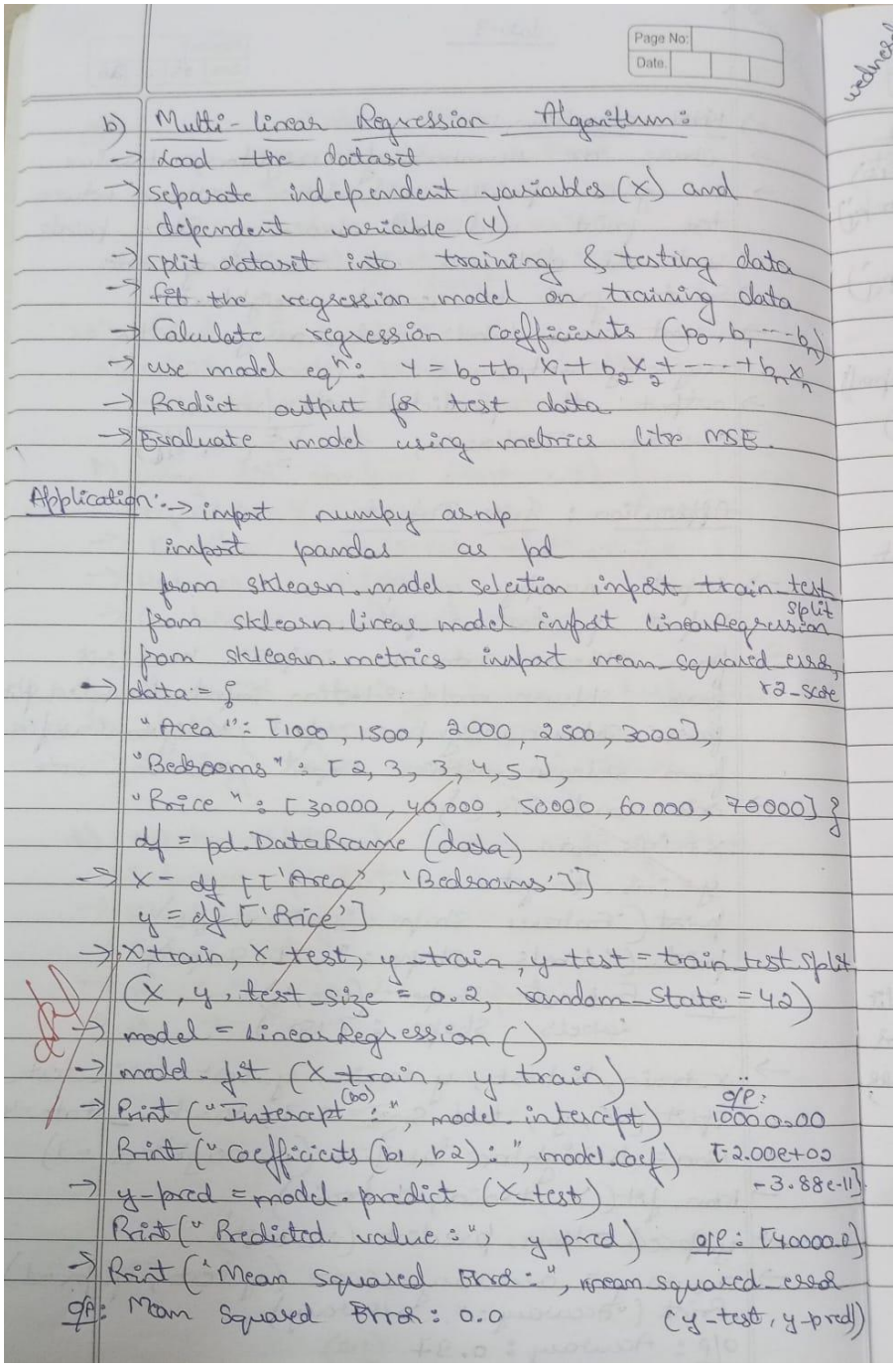
```
import pandas as pd

import numpy as np

data = pd.DataFrame({

    'Name': ['Asha', 'Ravi', 'Meena', 'Kiran', 'Ravi'],

    'Age': [20, 21, np.nan, 22, 21],

    'Marks': [85, 78, 92, np.nan, 78],
```

```
    'City': ['Bangalore', 'Mysore', 'Bangalore', 'Chennai', 'Mysore']
})
print("Original Data:")
print(data)
data = data.drop_duplicates()
data['Age'] = data['Age'].fillna(data['Age'].mean())
data['Marks'] = data['Marks'].fillna(data['Marks'].mean())
data['Age'] = (data['Age'] - data['Age'].min()) / (data['Age'].max() - data['Age'].min())
data = pd.get_dummies(data, columns=['City'])
print("\nProcessed Data:")
print(data)
```

Program 3

Implement Linear and Multi-Linear Regression algorithm using appropriate dataset

Screenshot



b) Multi-linear Regression Algorithm:

-> Load the dataset
-> Separate independent variables (X) and dependent variable (Y)
-> Split dataset into training & testing data
-> Fit the regression model on training data
-> Calculate regression coefficients ($b_0, b_1 \cdots b_n$)
-> use model eqn: $Y = b_0 + b_1 X_1 + b_2 X_2 + \cdots + b_n X_n$
-> Predict output for test data
-> Evaluate model using metrics like MSE.

Application:

```
-> import numpy as np
   import pandas as pd
   from sklearn.model_selection import train_test_split
   from sklearn.linear_model import LinearRegression
   from sklearn.metrics import mean_squared_error, r2_score
-> data = {
   "Area": [1000, 1500, 2000, 2500, 3000],
   "Bedrooms": [2, 3, 3, 4, 5],
   "Price": [30000, 40000, 50000, 60000, 70000] }
   df = pd.DataFrame(data)
-> X = df[['Area', 'Bedrooms']]
   y = df['Price']
-> X_train, X_test, y_train, y_test = train_test_split
   (X, y, test_size = 0.2, random_state = 42)
-> model = LinearRegression()
-> model.fit(X_train, y_train)
-> Print("Intercept (b0):", model.intercept)        o/p: 10000.00
   Print("Coefficients (b1, b2):", model.coef)      [2.00e+00
                                                     -3.88e-11]
-> y_pred = model.predict(X_test)
   Print("Predicted value:", y_pred)                o/p: [40000.0]
-> Print("Mean Squared Error:", mean_squared_error
   (y_test, y_pred))
o/p: Mean Squared Error: 0.0
```

Code:

```
from google.colab import files
uploaded = files.upload()


import numpy as np
import pandas as pd
df = pd.read_csv("Housing.csv")   # change name if needed
print("Dataset Preview:")
print(df.head())
binary_cols = [
    'mainroad', 'guestroom', 'basement',
    'hotwaterheating', 'airconditioning', 'prefarea'
]
for col in binary_cols:
    df[col] = df[col].map({'yes': 1, 'no': 0})
df['furnishingstatus'] = df['furnishingstatus'].map({
    'furnished': 2,
    'semi-furnished': 1,
    'unfurnished': 0
})
print("\nAfter Encoding:")
print(df.head())
y = df['price'].values
X = df.drop('price', axis=1).values
```

```
def normalize(X):
    return (X - X.mean(axis=0)) / X.std(axis=0)
X = normalize(X)


    def train_test_split(X, y, test_size=0.2):
    indices = np.arange(len(X))
    np.random.shuffle(indices)
    split = int(len(X) * (1 - test_size))
    return X[indices[:split]], X[indices[split:]], y[indices[:split]], y[indices[split:]]
X_train, X_test, y_train, y_test = train_test_split(X, y)
def add_bias(X):
    return np.c_[np.ones(X.shape[0]), X]
X_train = add_bias(X_train)
X_test = add_bias(X_test)
def train_linear_regression(X, y, lr=0.01, epochs=1000):
    m, n = X.shape
    weights = np.zeros(n)
    for _ in range(epochs):
        y_pred = np.dot(X, weights)
        error = y_pred - y
        gradient = (1/m) * np.dot(X.T, error)
        weights -= lr * gradient
    return weights
weights = train_linear_regression(X_train, y_train)
```

```
def predict(X, weights):
    return np.dot(X, weights)
y_pred = predict(X_test, weights)


def mse(y_true, y_pred):
    return np.mean((y_true - y_pred) ** 2)


def r2_score(y_true, y_pred):
    ss_total = np.sum((y_true - np.mean(y_true))**2)
    ss_residual = np.sum((y_true - y_pred)**2)
    return 1 - (ss_residual / ss_total)


print("\nFinal Results:")
print("Weights:", weights)
print("MSE:", mse(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

Dataset Preview:

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement \
0	13300000	7420	4	2	3	yes	no	no
1	12250000	8960	4	4	4	yes	no	no
2	12250000	9960	3	2	2	yes	no	yes
3	12215000	7500	4	2	2	yes	no	yes
4	11410000	7420	4	1	2	yes	yes	yes

	hotwaterheating	airconditioning	parking	prefarea	furnishingstatus
0	no	yes	2	yes	furnished
1	no	yes	3	no	furnished
2	no	no	2	yes	semi-furnished
3	no	yes	3	yes	furnished
4	no	yes	2	no	furnished

After Encoding:

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom \
0	13300000	7420	4	2	3	1	0
1	12250000	8960	4	4	4	1	0
2	12250000	9960	3	2	2	1	0
3	12215000	7500	4	2	2	1	0
4	11410000	7420	4	1	2	1	1

	basement	hotwaterheating	airconditioning	parking	prefarea \
0	0	0	1	2	1
1	0	0	1	3	0
2	1	0	0	2	1
3	1	0	1	3	1
4	1	0	1	2	0

furnishingstatus

0	2
1	2
2	1
3	2
4	2

Final Results:

Weights: [4769696.4307444 530083.45838771 89000.01914967 492503.02118445 413478.07121193 126632.01009817 127713.38381088 165395.56832441 155995.94227929 375830.53960283 227720.57883313 271817.96131604 194359.21025863]

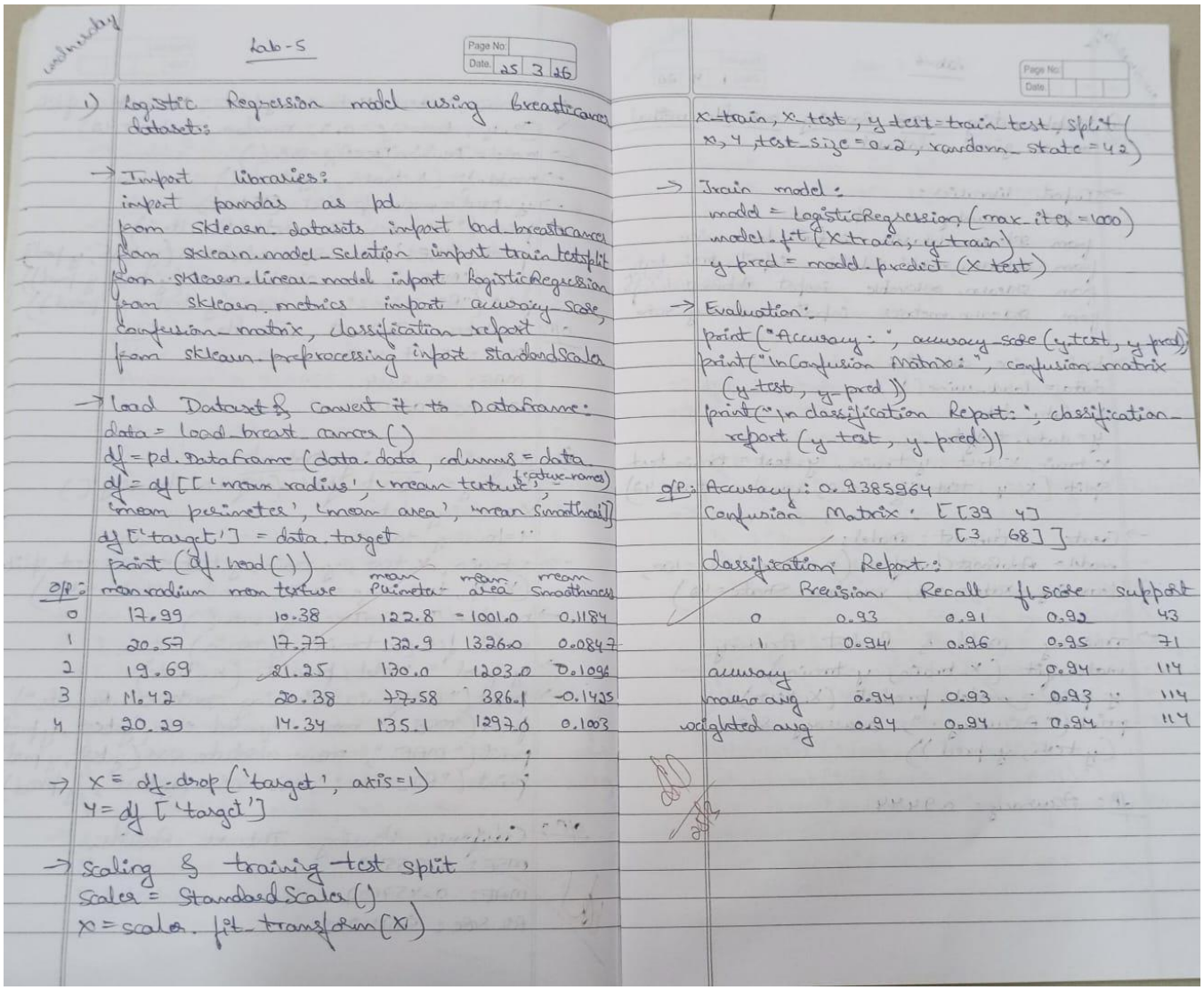
MSE: 917397282501.2964

R2 Score: 0.7151269592549224

Program 4

Build Logistic Regression Model for a given dataset

Screenshot



Lab-5

Page No:
Date: 25 3 26

1) Logistic Regression model using Breastcancer datasets:

→ Import libraries:

```
import pandas as pd
from sklearn.datasets import load_breastcancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.preprocessing import StandardScaler
```

→ Load Dataset & convert it to Dataframe:

```
data = load_breast_cancer()
df = pd.DataFrame(data.data, columns = data.feature_names)
df = df[['mean radius', 'mean texture', 'mean perimeter', 'mean area', 'mean smoothness']]
df['target'] = data.target
print(df.head())
```

o/p:

	mean radius	mean texture	mean Perimeter	mean area	mean Smoothness
0	17.99	10.38	122.8	1001.0	0.1184
1	20.57	17.77	132.9	1326.0	0.0847
2	19.69	21.25	130.0	1203.0	0.1096
3	11.42	20.38	77.58	386.1	0.1425
4	20.29	14.34	135.1	1297.0	0.1003

→ x = df.drop('target', axis=1)
y = df['target']

→ Scaling & training test split

```
scaler = StandardScaler()
x = scaler.fit_transform(x)
x_train, x_test, y_test = train_test_split(x, y, test_size = 0.2, random_state = 42)
```

→ Train model:

```
model = LogisticRegression(max_iter=1000)
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
```

→ Evaluation:

```
print("Accuracy: ", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix: ", confusion_matrix(y_test, y_pred))
print("\n classification Report: ", classification_report(y_test, y_pred))
```

o/p: Accuracy: 0.9385964
Confusion Matrix: [[39 4]
[3 68]]

classification Report:

	Precision	Recall	f1 score	support
0	0.93	0.91	0.92	43
1	0.94	0.96	0.95	71
accuracy			0.94	114
macro avg	0.94	0.93	0.93	114
weighted avg	0.94	0.94	0.94	114

Code:

```
import pandas as pd

from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
from sklearn.preprocessing import StandardScaler



data = load_breast_cancer()

df = pd.DataFrame(data.data, columns=data.feature_names)

df['target'] = data.target

print(df.head())


   mean radius  mean texture  mean perimeter  mean area  mean smoothness  \

0        17.99         10.38          122.80     1001.0          0.11840

1        20.57         17.77          132.90     1326.0          0.08474

2        19.69         21.25          130.00     1203.0          0.10960

3        11.42         20.38           77.58      386.1          0.14250

4        20.29         14.34          135.10     1297.0          0.10030


   mean compactness  mean concavity  mean concave points  mean symmetry  \

0           0.27760          0.3001              0.14710         0.2419

1           0.07864          0.0869              0.07017         0.1812

2           0.15990          0.1974              0.12790         0.2069

3           0.28390          0.2414              0.10520         0.2597

4           0.13280          0.1980              0.10430         0.1809


   mean fractal dimension  ...  worst texture  worst perimeter  worst area  \

0                 0.07871  ...          17.33           184.60      2019.0
```

```
1          0.05667  ...      23.41       158.80    1956.0
2          0.05999  ...      25.53       152.50    1709.0
3          0.09744  ...      26.50        98.87     567.7
4          0.05883  ...      16.67       152.20    1575.0

   worst smoothness  worst compactness  worst concavity  worst concave points  \
0            0.1622             0.6656           0.7119                0.2654
1            0.1238             0.1866           0.2416                0.1860
2            0.1444             0.4245           0.4504                0.2430
3            0.2098             0.8663           0.6869                0.2575
4            0.1374             0.2050           0.4000                0.1625

   worst symmetry  worst fractal dimension  target
0          0.4601                  0.11890       0
1          0.2750                  0.08902       0
2          0.3613                  0.08758       0
3          0.6638                  0.17300       0
4          0.2364                  0.07678       0
df = df[['mean radius', 'mean texture', 'mean perimeter', 'mean area', 'mean smoothness']]
df['target'] = data.target
print(df.head())

   mean radius  mean texture  mean perimeter  mean area  mean smoothness  \
0        17.99         10.38          122.80     1001.0          0.11840
```

1	20.57	17.77	132.90	1326.0	0.08474
2	19.69	21.25	130.00	1203.0	0.10960
3	11.42	20.38	77.58	386.1	0.14250
4	20.29	14.34	135.10	1297.0	0.10030

	target
0	0
1	0
2	0
3	0
4	0

```
X = df.drop('target', axis=1)
y = df['target']
scaler = StandardScaler()
X = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
LogisticRegression(max_iter=1000)
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

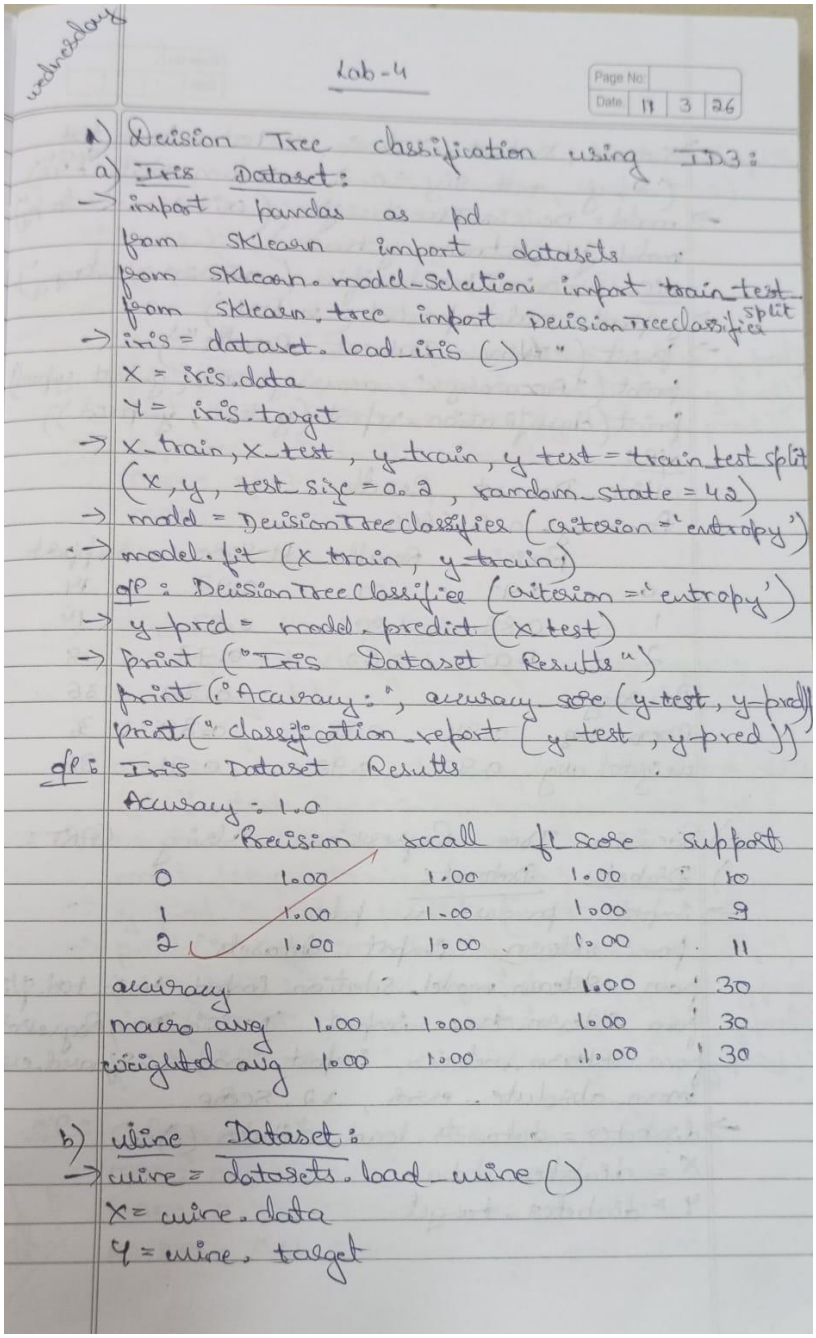
```
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
Accuracy: 0.9385964912280702

Confusion Matrix:
 [[39  4]
 [ 3 68]]

Classification Report:
              precision    recall  f1-score   support

           0       0.93      0.91      0.92        43
           1       0.94      0.96      0.95        71

    accuracy                           0.94       114
   macro avg       0.94      0.93      0.93       114
weighted avg       0.94      0.94      0.94       114
```

Program 5

Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.

Screenshot



Lab-4

Page No:
Date 11 3 26

1) Decision Tree classification using ID3:

a) Iris Dataset:

```
-> import pandas as pd
   from sklearn import datasets
   from sklearn.model_selection import train_test_split
   from sklearn.tree import DecisionTreeClassifier
-> iris = dataset.load_iris()
   X = iris.data
   Y = iris.target
-> X_train, X_test, y_train, y_test = train_test_split
   (X, y, test_size = 0.2, random_state = 42)
-> model = DecisionTreeClassifier (criterion='entropy')
-> model.fit (X_train, y_train)
   o/p: DecisionTreeClassifier (criterion='entropy')
-> y_pred = model.predict (X_test)
-> print ("Iris Dataset Results")
   print ("Accuracy:", accuracy_score (y_test, y_pred))
   print ("classification_report (y_test, y_pred))
```

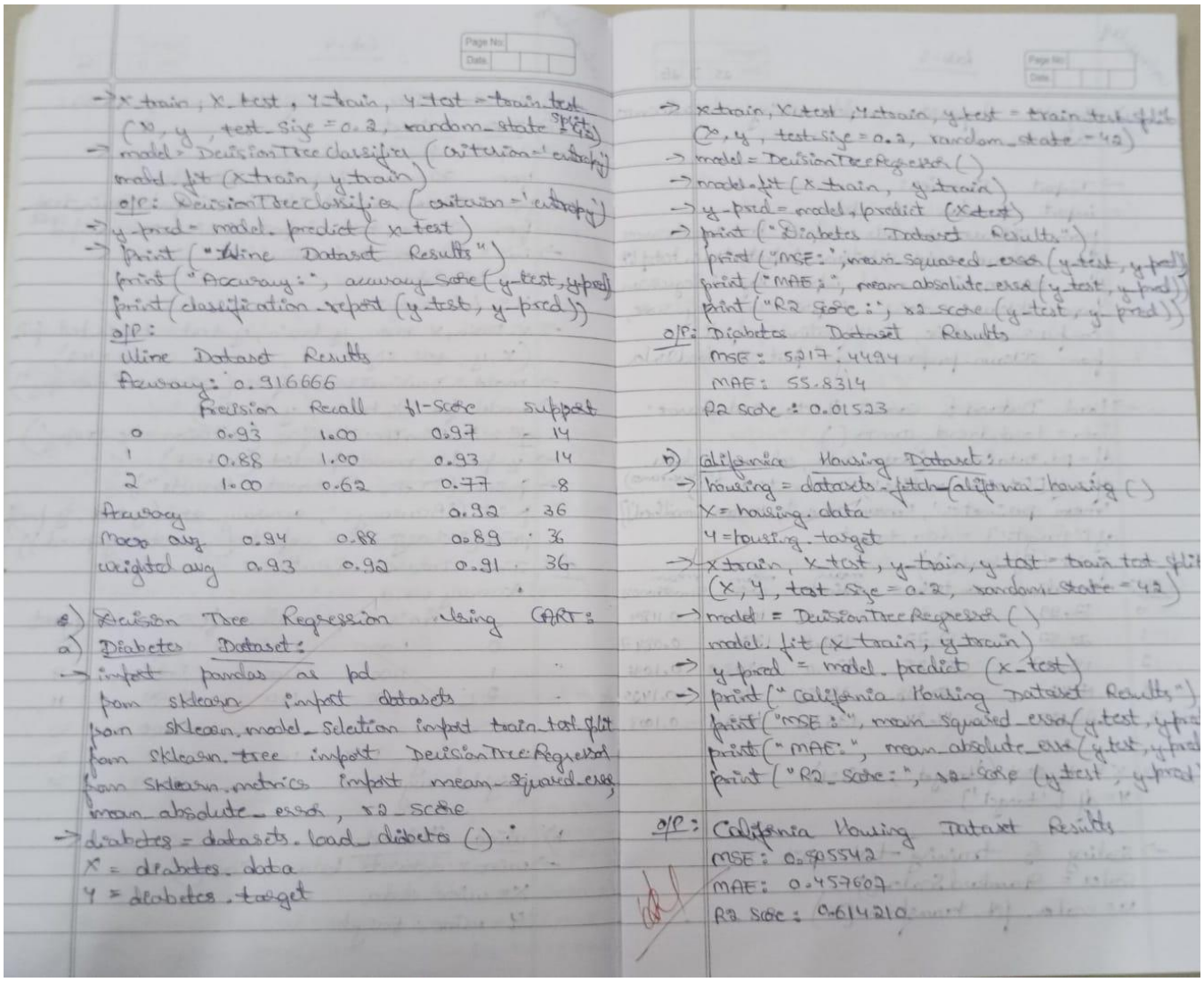
o/p: Iris Dataset Results

Accuracy: 1.0

	precision	recall	f1 score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	9
2	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

b) Wine Dataset:

```
-> wine = datasets.load_wine()
   X = wine.data
   Y = wine.target
```



Code:

```
import pandas as pd

from sklearn import datasets

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import accuracy_score, classification_report
```

```
iris = datasets.load_iris()

X = iris.data

y = iris.target

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)

model = DecisionTreeClassifier(criterion='entropy')

model.fit(X_train, y_train)

DecisionTreeClassifier(criterion='entropy')

y_pred = model.predict(X_test)

print("IRIS DATASET RESULTS")

print("Accuracy:", accuracy_score(y_test, y_pred))

print(classification_report(y_test, y_pred))
```

```
IRIS DATASET RESULTS

Accuracy: 1.0

              precision    recall  f1-score   support


           0       1.00      1.00      1.00        10

           1       1.00      1.00      1.00         9

           2       1.00      1.00      1.00        11


    accuracy                           1.00        30

   macro avg       1.00      1.00      1.00        30
```

```
weighted avg       1.00      1.00      1.00        30
```

```
wine = datasets.load_wine()
X = wine.data
y = wine.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = DecisionTreeClassifier(criterion='entropy')
model.fit(X_train, y_train)
DecisionTreeClassifier(criterion='entropy')
y_pred = model.predict(X_test)
print("\nWINE DATASET RESULTS")
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

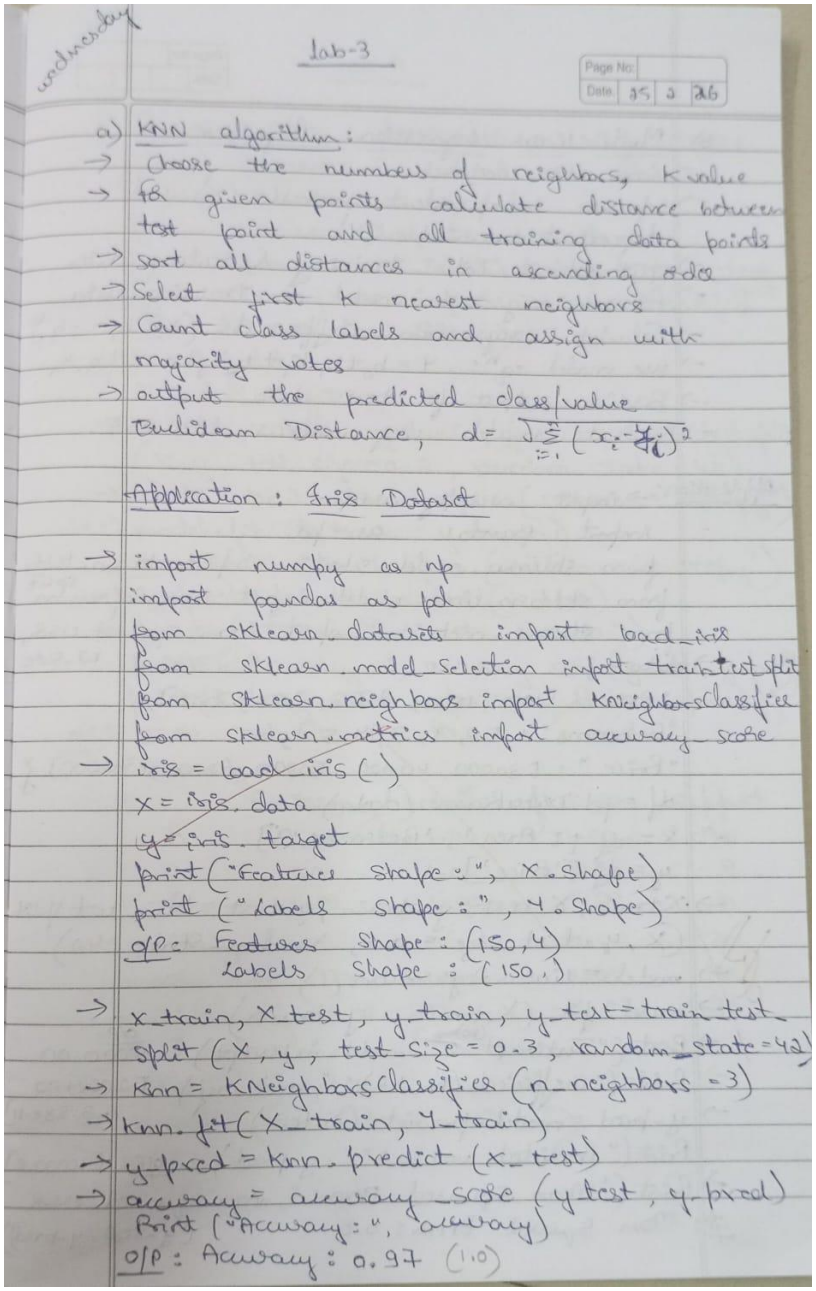
```
WINE DATASET RESULTS
Accuracy: 0.9166666666666666
              precision    recall  f1-score   support

           0       0.93      1.00      0.97        14
           1       0.88      1.00      0.93        14
           2       1.00      0.62      0.77         8
```

accuracy			0.92	36
macro avg	0.94	0.88	0.89	36
weighted avg	0.93	0.92	0.91	36

Program 6

Build KNN Classification model for a given dataset.

Screenshot



lab-3

a) KNN algorithm:

→ Choose the numbers of neighbors, K value
→ for given points calculate distance between test point and all training data points
→ sort all distances in ascending order
→ Select first K nearest neighbors
→ Count class labels and assign with majority votes
→ output the predicted class/value

Euclidean Distance, $d = \sqrt{\sum_{i=1} (x_i - y_i)^2}$

Application : Iris Dataset

```
→ import numpy as np
  import pandas as pd
  from sklearn.datasets import load_iris
  from sklearn.model_selection import train_test_split
  from sklearn.neighbors import KNeighborsClassifier
  from sklearn.metrics import accuracy_score
→ iris = load_iris()
  X = iris.data
  y = iris.target
  print("Features shape :", X.shape)
  print("Labels shape :", y.shape)
  o/p: Features shape : (150,4)
       Labels shape : (150,)
→ X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.3, random_state=42)
→ knn = KNeighborsClassifier(n_neighbors = 3)
→ knn.fit(X_train, y_train)
→ y_pred = knn.predict(X_test)
→ accuracy = accuracy_score(y_test, y_pred)
  Print("Accuracy:", accuracy)
  o/p : Accuracy : 0.97 (1.0)
```

Code:

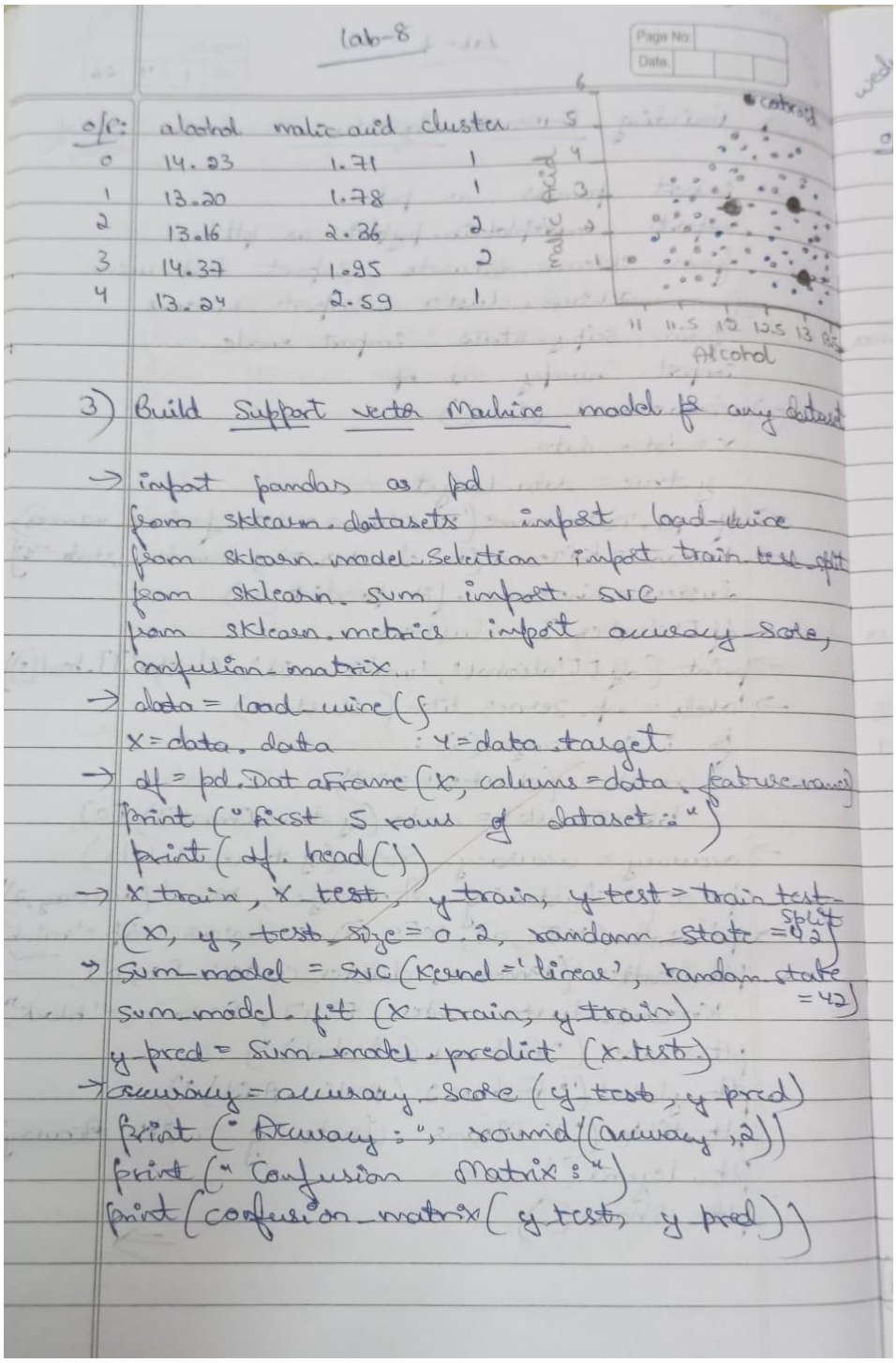
```
import numpy as np
```

```
import pandas as pd

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score

iris = load_iris()

X = iris.data        # Features

y = iris.target      # Labels

print("Features shape:", X.shape)

print("Labels shape:", y.shape)

Features shape: (150, 4)

Labels shape: (150,)

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.3, random_state=42

)

knn = KNeighborsClassifier(n_neighbors=3)

knn.fit(X_train, y_train)

KNeighborsClassifier(n_neighbors=3)

y_pred = knn.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

Accuracy: 1.0
```

Program 7

Build Support vector machine model for a given dataset

Screenshot



lab-8

o/p:	alcohol	malic acid	cluster
0	14.23	1.71	1
1	13.20	1.78	1
2	13.16	2.36	2
3	14.37	1.95	2
4	13.24	2.59	1

3) Build Support vector Machine model for any dataset

```
→ import pandas as pd
  from sklearn.datasets import load_wine
  from sklearn.model_selection import train_test_split
  from sklearn.svm import SVC
  from sklearn.metrics import accuracy_score,
  confusion_matrix
→ data = load_wine()
  X=data.data        y=data.target
→ df = pd.DataFrame(X, columns=data.feature_names)
  print("first 5 rows of dataset:")
  print(df.head())
→ X_train, X_test, y_train, y_test = train_test_split
  (X, y, test_size = 0.2, random_state =42)
→ svm_model = SVC(kernel='linear', random_state
  =42)
  svm_model.fit(X_train, y_train)
  y_pred = svm_model.predict(X_test)
→ accuracy = accuracy_score(y_test, y_pred)
  print("Accuracy:", round(accuracy,2))
  print("Confusion Matrix:")
  print(confusion_matrix(y_test, y_pred))
```

Code:

```
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix
data=load_wine()
X=data.data
y=data.target
df=pd.DataFrame(X,columns=data.feature_names)
print("First 5 rows of dataset:")
print(df.head())
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)
svm_model=SVC(kernel='linear',random_state=42)
svm_model.fit(X_train,y_train)
y_pred=svm_model.predict(X_test)
accuracy=accuracy_score(y_test,y_pred)
print("\nAccuracy:",round(accuracy,2))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test,y_pred))
```

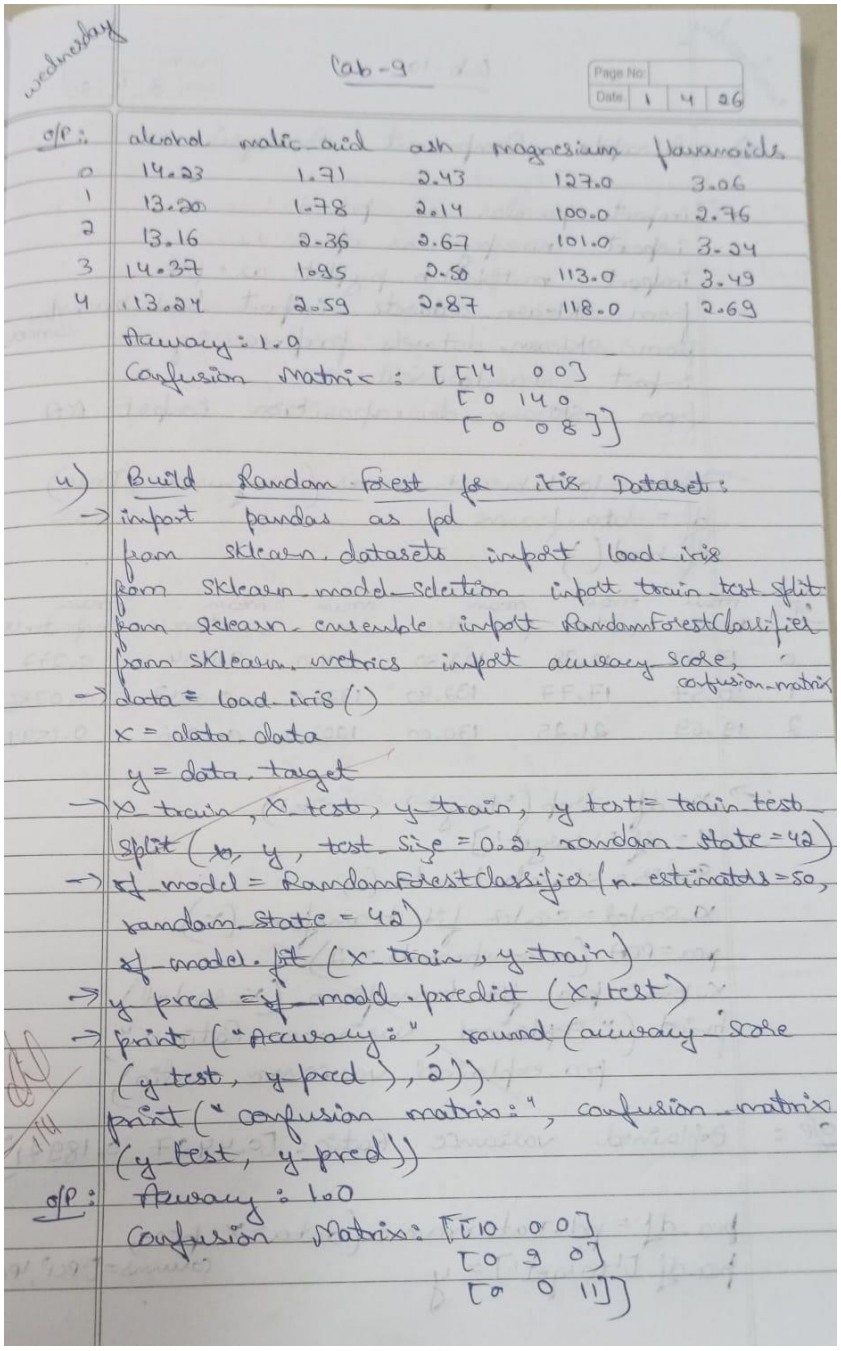
```
First 5 rows of dataset:
   alcohol  malic_acid   ash  alcalinity_of_ash  magnesium  total_phenols  \
0    14.23        1.71  2.43               15.6      127.0           2.80
1    13.20        1.78  2.14               11.2      100.0           2.65
```

```
2    13.16       2.36  2.67            18.6     101.0         2.80

3    14.37       1.95  2.50            16.8     113.0         3.85

4    13.24       2.59  2.87            21.0     118.0         2.80


   flavanoids  nonflavanoid_phenols  proanthocyanins  color_intensity   hue  \

0       3.06                 0.28            2.29            5.64  1.04

1       2.76                 0.26            1.28            4.38  1.05

2       3.24                 0.30            2.81            5.68  1.03

3       3.49                 0.24            2.18            7.80  0.86

4       2.69                 0.39            1.82            4.32  1.04


   od280/od315_of_diluted_wines  proline

0                          3.92   1065.0

1                          3.40   1050.0

2                          3.17   1185.0

3                          3.45   1480.0

4                          2.93    735.0


Accuracy: 1.0

Confusion Matrix:

[[14  0  0]

 [ 0 14  0]

 [ 0  0  8]]
```

Program 8

Implement Random forest ensemble method on a given dataset.

Screenshot



Lab-9

Page No:
Date: 1 4 26

o/p:

	alcohol	malic_acid	ash	magnesium	flavanoids
0	14.23	1.71	2.43	127.0	3.06
1	13.20	1.78	2.14	100.0	2.76
2	13.16	2.36	2.67	101.0	3.24
3	14.37	1.95	2.50	113.0	3.49
4	13.24	2.59	2.87	118.0	2.69

Accuracy: 1.0
Confusion matrix: [[14 0 0]
[0 14 0
[0 0 8]]

4) Build Random Forest for iris Dataset:
→ import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
→ data = load_iris()
x = data.data
y = data.target
→ x_train, x_test, y_train, y_test = train_test_split (x, y, test_size = 0.2, random_state = 42)
→ rf_model = RandomForestClassifier (n_estimators = 50, random_state = 42)
rf_model.fit (x_train, y_train)
→ y_pred = rf_model.predict (x_test)
→ print ("Accuracy:", round (accuracy_score (y_test, y_pred), 2))
print ("confusion matrix:", confusion_matrix (y_test, y_pred))

o/p: Accuracy: 1.0
Confusion Matrix: [[10 0 0]
[0 9 0]
[0 0 11]]

Code:

```
from sklearn.datasets import load_iris
```

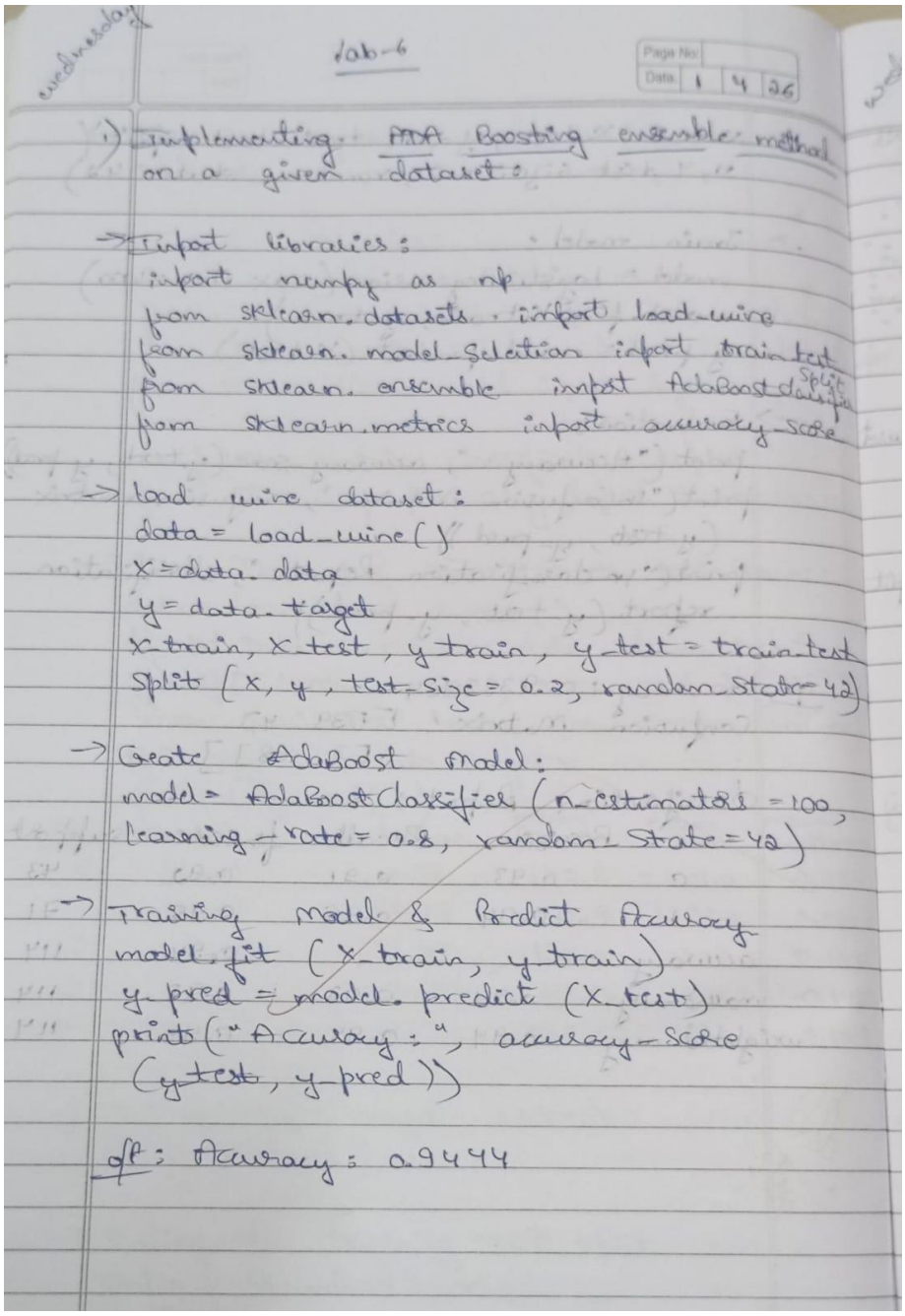
```
from sklearn.model_selection import train_test_split

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score,confusion_matrix

data=load_iris()

X=data.data

y=data.target

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)

rf_model=RandomForestClassifier(n_estimators=50,random_state=42)

rf_model.fit(X_train,y_train)

y_pred=rf_model.predict(X_test)

print("Accuracy:",round(accuracy_score(y_test,y_pred),2))

print("Confusion Matrix:\n",confusion_matrix(y_test,y_pred))
```

```
Accuracy: 1.0

Confusion Matrix:

 [[10  0  0]

 [ 0  9  0]

 [ 0  0 11]]
```

Program 9

Implement Boosting ensemble method on a given dataset.

Screenshot



Lab-6

Page No:
Date: 1 4 26

1) Implementing AdA Boosting ensemble method on a given dataset:

→ Import libraries:

```
import numpy as np
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score
```

→ load wine dataset:

```
data = load_wine()
X = data.data
y = data.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state=42)
```

→ Create AdaBoost model:

```
model = AdaBoostClassifier(n_estimators =100, learning_rate = 0.8, random_state=42)
```

→ Training model & Predict Accuracy

```
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

op: Accuracy: 0.9444

Code:

```
import numpy as np

from sklearn.datasets import load_wine
```

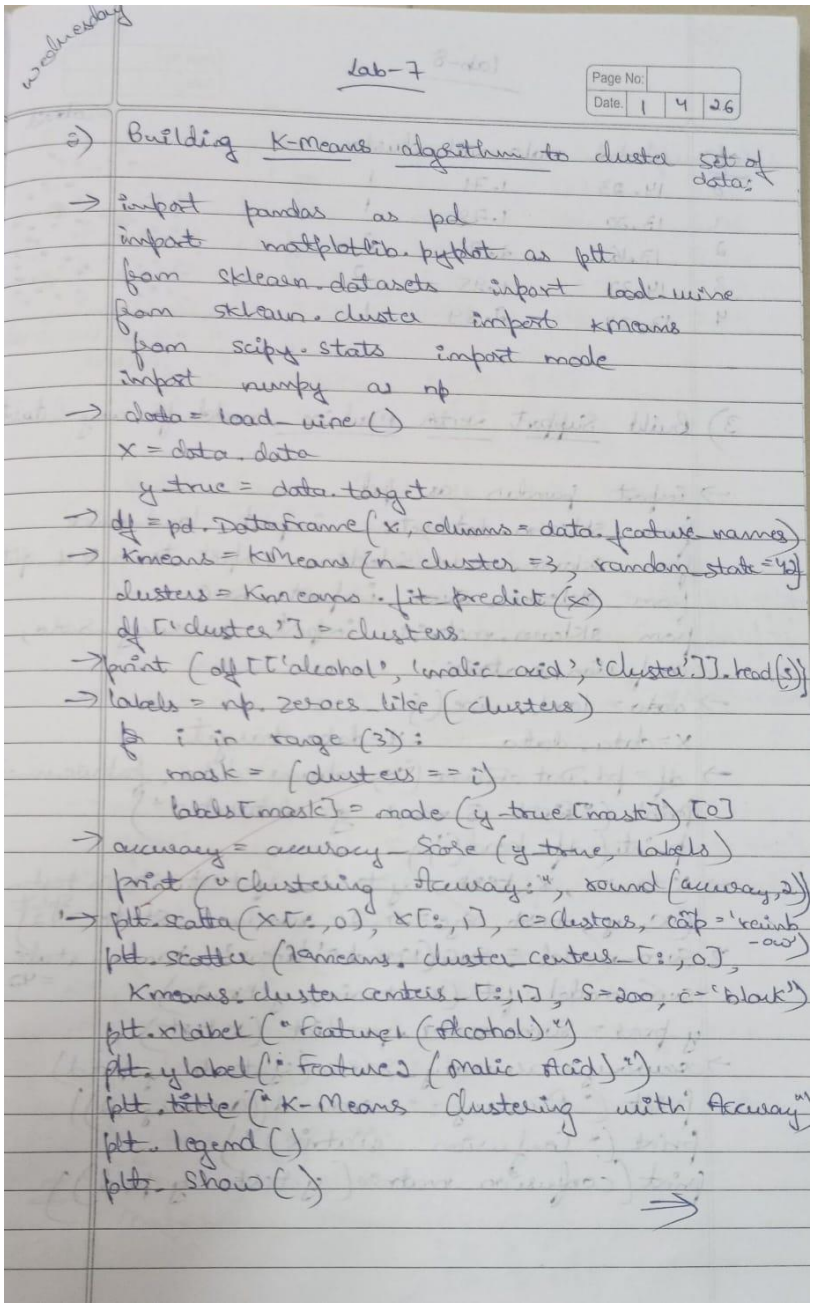
```
from sklearn.model_selection import train_test_split

from sklearn.ensemble import AdaBoostClassifier

from sklearn.metrics import accuracy_score

data=load_wine()

X=data.data

y=data.target

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)

model=AdaBoostClassifier(n_estimators=100,learning_rate=0.8,random_state=42)

model.fit(X_train,y_train)

y_pred=model.predict(X_test)

print("Accuracy:",accuracy_score(y_test,y_pred))
```

```
Accuracy: 0.9444444444444444
```

Program 10

Build k-Means algorithm to cluster a set of data stored in a .CSV file.

Screenshot



Lab-7

Page No:
Date. 1 4 26

2) Building K-Means algorithm to cluster set of data:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.cluster import kmeans
from scipy.stats import mode
import numpy as np
data = load_wine()
x = data.data
y_true = data.target
df = pd.DataFrame(x, columns = data.feature_names)
Kmeans = KMeans(n_cluster =3, random_state=42)
clusters = Kmeans.fit_predict(x)
df['cluster'] = clusters
print(df[['alcohol', 'malic_acid', 'cluster']].head(5))
labels = np.zeroes_like(clusters)
for i in range(3):
    mask = (clusters == i)
    labels[mask] = mode(y_true[mask])[0]
accuracy = accuracy_score(y_true, labels)
print("clustering Accuracy:", round(accuracy,2))
plt.scatter(x[:,0], x[:,1], c=clusters, cmap='rainbow')
plt.scatter(kmeans.cluster_centers_[:,0],
  Kmeans.cluster_centers_[:,1], s=200, c='black')
plt.xlabel("Feature1 (Alcohol)")
plt.ylabel("Feature2 (Malic Acid)")
plt.title("K-Means Clustering with Accuracy")
plt.legend()
plt.show()
```

Code:

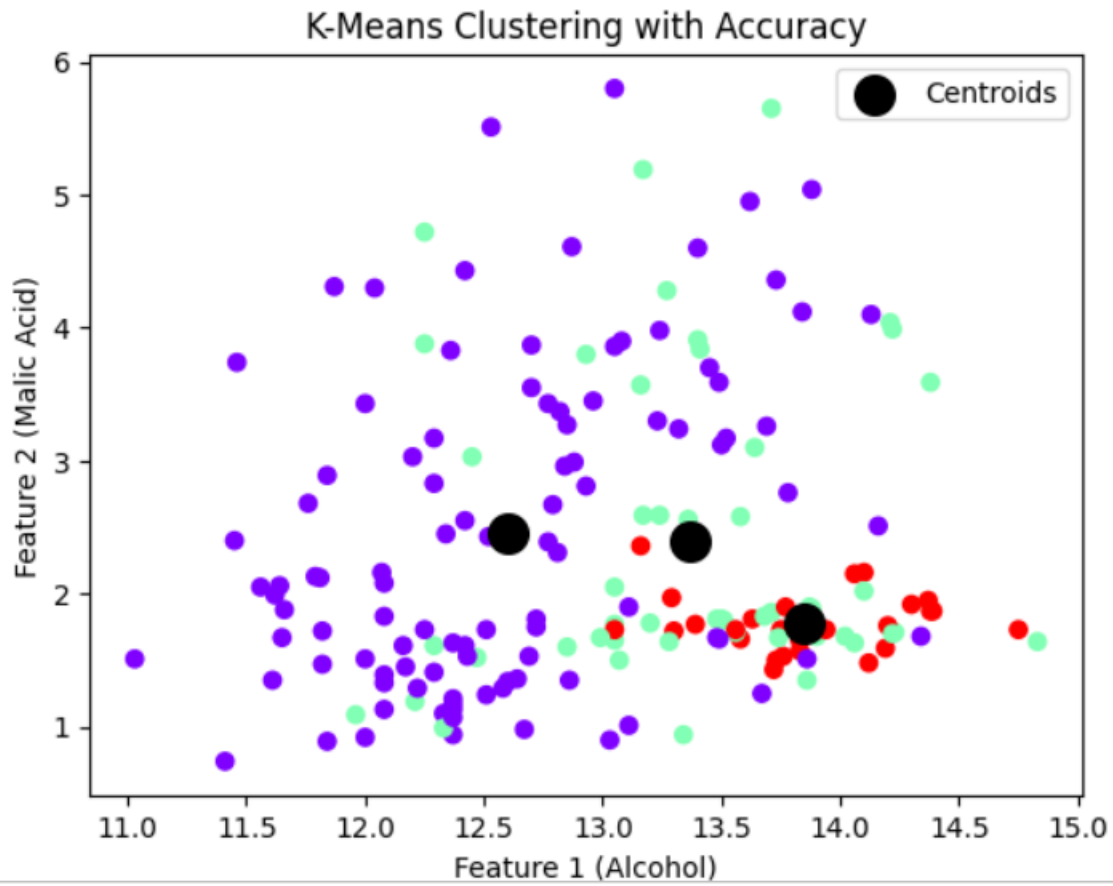
```
import pandas as pd

import matplotlib.pyplot as plt
```

```
from sklearn.datasets import load_wine

from sklearn.cluster import KMeans

from sklearn.metrics import accuracy_score

from scipy.stats import mode

import numpy as np

data=load_wine()

X=data.data

y_true=data.target

df=pd.DataFrame(X,columns=data.feature_names)

kmeans=KMeans(n_clusters=3,random_state=42)

clusters=kmeans.fit_predict(X)

df['Cluster']=clusters

print(df[['alcohol','malic_acid','Cluster']].head(10))

labels=np.zeros_like(clusters)

for i in range(3):

    mask=(clusters==i)

    labels[mask]=mode(y_true[mask])[0]

accuracy=accuracy_score(y_true,labels)

print("Clustering Accuracy:",round(accuracy,2))

plt.scatter(X[:,0],X[:,1],c=clusters,cmap='rainbow')

plt.scatter(kmeans.cluster_centers_[:,0],kmeans.cluster_centers_[:,1],s=200,c='black',label='Centroid
s')

plt.xlabel("Feature 1 (Alcohol)")

plt.ylabel("Feature 2 (Malic Acid)")

plt.title("K-Means Clustering with Accuracy")
```

plt.legend()

plt.show()

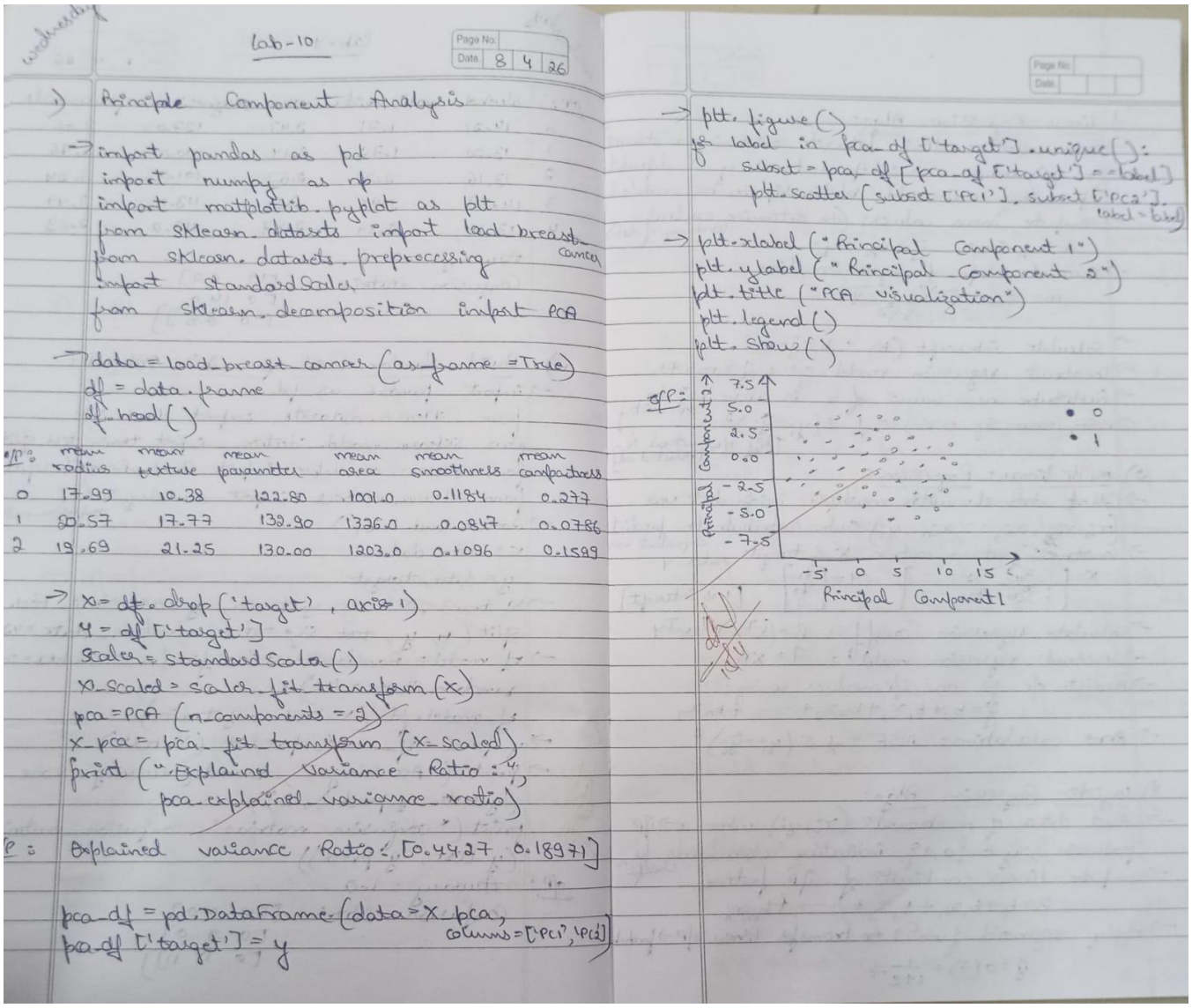
```
   alcohol  malic_acid  Cluster
0    14.23        1.71        1
1    13.20        1.78        1
2    13.16        2.36        2
3    14.37        1.95        2
4    13.24        2.59        1
5    14.20        1.76        2
6    14.39        1.87        2
7    14.06        2.15        2
8    14.83        1.64        1
9    13.86        1.35        1
Clustering Accuracy: 0.69
```



Program 11

Implement Dimensionality reduction using Principal Component Analysis (PCA) method.

Screenshot



lab-10

Page No:
Date 8 4 26

1) Principle Component Analysis

```
-> import pandas as pd
   import numpy as np
   import matplotlib.pyplot as plt
   from sklearn.datasets import load_breast_cancer
   from sklearn.datasets.preprocessing
   import StandardScaler
   from sklearn.decomposition import PCA

-> data = load_breast_cancer(as_frame=True)
   df = data.frame
   df.head()
```

o/p:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness
0	17.99	10.38	122.80	1001.0	0.1184	0.277
1	20.57	17.77	132.90	1326.0	0.0847	0.0786
2	19.69	21.25	130.00	1203.0	0.1096	0.1599

```
-> x = df.drop('target', axis=1)
   y = df['target']
   scaler = StandardScaler()
   x_scaled = scaler.fit_transform(x)
   pca = PCA(n_components=2)
   x_pca = pca.fit_transform(x_scaled)
   print("Explained Variance Ratio:",
         pca.explained_variance_ratio)
```

o/p: Explained variance Ratio: [0.4427 0.18971]

```
   pca_df = pd.DataFrame(data=x_pca,
                         columns=['PC1','PC2'])
   pca_df['target'] = y
```

```
-> plt.figure()
   for label in pca_df['target'].unique():
       subset = pca_df[pca_df['target'] == label]
       plt.scatter(subset['PC1'], subset['PC2'],
                   label=label)
-> plt.xlabel("Principal Component 1")
   plt.ylabel("Principal Component 2")
   plt.title("PCA visualization")
   plt.legend()
   plt.show()
```

o/p:

Code:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from google.colab import files
```

```
from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import PCA

uploaded=files.upload()

df=pd.read_csv("housing.csv")

print(df.head())

binary_cols=['mainroad','guestroom','basement','hotwaterheating','airconditioning','prefarea']

for col in binary_cols:

    df[col]=df[col].map({'yes':1,'no':0})

df['furnishingstatus']=df['furnishingstatus'].map({'furnished':2,'semi-furnished':1,'unfurnished':0})

X=df.drop('price',axis=1)

scaler=StandardScaler()

X_scaled=scaler.fit_transform(X)

pca=PCA(n_components=2)

X_pca=pca.fit_transform(X_scaled)

print(pca.explained_variance_ratio_)

pca_df=pd.DataFrame(X_pca,columns=['PC1','PC2'])

print(pca_df.head())

plt.figure(figsize=(7,5))

plt.scatter(pca_df['PC1'],pca_df['PC2'],alpha=0.6,color='blue')

plt.xlabel("Principal Component 1")

plt.ylabel("Principal Component 2")

plt.title("PCA - 2D Projection of Data")

plt.grid()

plt.show()
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness \
0	17.99	10.38	122.80	1001.0	0.11840
1	20.57	17.77	132.90	1326.0	0.08474
2	19.69	21.25	130.00	1203.0	0.10960
3	11.42	20.38	77.58	386.1	0.14250
4	20.29	14.34	135.10	1297.0	0.10030

	mean compactness	mean concavity	mean concave points	mean symmetry \
0	0.27760	0.3001	0.14710	0.2419
1	0.07864	0.0869	0.07017	0.1812
2	0.15990	0.1974	0.12790	0.2069
3	0.28390	0.2414	0.10520	0.2597
4	0.13280	0.1980	0.10430	0.1809

	mean fractal dimension	...	worst texture	worst perimeter	worst area \
0	0.07871	...	17.33	184.60	2019.0
1	0.05667	...	23.41	158.80	1956.0
2	0.05999	...	25.53	152.50	1709.0
3	0.09744	...	26.50	98.87	567.7
4	0.05883	...	16.67	152.20	1575.0

	worst smoothness	worst compactness	worst concavity	worst concave points \
0	0.1622	0.6656	0.7119	0.2654

1	0.1238	0.1866	0.2416	0.1860
2	0.1444	0.4245	0.4504	0.2430
3	0.2098	0.8663	0.6869	0.2575
4	0.1374	0.2050	0.4000	0.1625

	worst symmetry	worst fractal dimension	target
0	0.4601	0.11890	0
1	0.2750	0.08902	0
2	0.3613	0.08758	0
3	0.6638	0.17300	0
4	0.2364	0.07678	0

[5 rows x 31 columns]

	mean radius	mean texture	mean perimeter	mean area	mean smoothness \
0	17.99	10.38	122.80	1001.0	0.11840
1	20.57	17.77	132.90	1326.0	0.08474
2	19.69	21.25	130.00	1203.0	0.10960
3	11.42	20.38	77.58	386.1	0.14250
4	20.29	14.34	135.10	1297.0	0.10030

	target
0	0
1	0
2	0

3 0

4 0

Accuracy: 0.9385964912280702

Confusion Matrix:

```
[[39  4]
 [ 3 68]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.93	0.91	0.92	43
1	0.94	0.96	0.95	71
accuracy			0.94	114
macro avg	0.94	0.93	0.93	114
weighted avg	0.94	0.94	0.94	114